МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Лабораторная работа №2

Дисциплина: Эволюционное программирование и генетические алгоритмы

Студент: Горбунов Сергей Михайлович
Группа: Б24-507
Вариант: V = 12
Дата: 2026 г.

Москва, 2026

Содержание

1 Постановка задачи и индивидуальный вариант **3**
1.1 Расчёт индивидуального варианта . . . 3
1.2 Постановка задачи оптимизации . . . 3

2 Формальное описание задачи **3**
2.1 Пространство решений . . . 3
2.2 Целевая функция . . . 4
2.3 Ограничения . . . 4
2.4 Теоретический ориентир . . . 4

3 Представление особи **4**
3.1 Генотип . . . 4
3.2 Фенотип . . . 4
3.3 Процедура декодирования . . . 4
3.4 Функция приспособленности . . . 5
3.5 Примеры допустимых и недопустимых решений . . . 5

4 Описание генетического алгоритма **5**
4.1 Общая структура . . . 5
4.2 Параметры алгоритма . . . 5
4.3 Инициализация . . . 6
4.4 Турнирная селекция . . . 6
4.5 Кроссинговер OX (Order Crossover) . . . 6
4.6 Мутация . . . 6
4.7 Ремонт . . . 6
4.8 Элитизм . . . 6
4.9 Критерий остановки . . . 6

5 Блок-схема алгоритма **7**

6 Обоснование выбора операторов **7**
6.1 Почему перестановка? . . . 7
6.2 Почему OX? . . . 7
6.3 Почему swap и inversion? . . . 7
6.4 Почему штрафы и ремонт? . . . 8
6.5 Почему элитизм? . . . 8

7 Результаты экспериментов **8**
7.1 Пробный запуск (seed = 42) . . . 8
7.2 Серия из 20 независимых запусков . . . 8
7.3 Сравнение двух операторов мутации . . . 8
7.4 Сравнение штрафов и ремонта . . . 8
7.5 Сравнение с жадной эвристикой и случайным поиском . . . 8

8 Анализ результатов **10**
8.1 Статистические показатели (20 запусков) . . . 10
8.2 Интерпретация . . . 10
8.3 Динамика сходимости . . . 10
8.4 Сравнение с теоретическим ориентиром . . . 10

9 Содержательные выводы **10**

10 Почему результат следует считать хорошим? **11**

11 Соответствие требованиям задания **11**

12 Заключение **12**

1 Постановка задачи и индивидуальный вариант

1.1 Расчёт индивидуального варианта

Индивидуальный вариант вычисляется по формуле из методических указаний:

$$V = 1 + ((17N + 7Q + Y) \bmod 20), \tag{1}$$

где:

- $N = 5$ — порядковый номер студента в группе;
- $Q = 0$ — индекс группы 507;
- $Y = 26$ — две последние цифры года (осенний семестр 2026).

Подставляем:

$$V = 1 + ((17 \cdot 5 + 7 \cdot 0 + 26) \bmod 20) = 1 + (111 \bmod 20) = 1 + 11 = 12. \tag{2}$$

Итоговый вариант: $V = 12$.

По таблице индивидуальных вариантов вариант 12 — это задача:

> *Маршрут доставки по 15–25 точкам с ограничением времени. Представление: перестановка + штрафы/ремонт.*

1.2 Постановка задачи оптимизации

Требуется найти маршрут доставки, минимизирующий суммарные затраты:

$$\min_{\pi \in S_n} F(\pi), \tag{3}$$

где π — перестановка $n = 20$ клиентов, S_n — множество всех перестановок, а $F(\pi)$ — целевая функция (см. §2.2).

2 Формальное описание задачи

2.1 Пространство решений

Курьер стартует из депо (индекс 0), должен посетить $n = 20$ клиентов ровно по одному разу и вернуться в депо.

Решение — перестановка:

$$\pi = (\pi_1, \pi_2, \ldots, \pi_n), \quad \pi_k \in \{1, \ldots, n\}, \quad \pi_i \neq \pi_j. \tag{4}$$

Мощность пространства поиска:

$$|S_n| = n! = 20! \approx 2.43 \cdot 10^{18}. \tag{5}$$

Полный перебор невозможен.

2.2 Целевая функция

$$F(\pi) = \underbrace{\sum_{k=1}^{n} d_{\pi_{k-1},\pi_k} + d_{\pi_n,0}}_{\text{длина маршрута}} + \alpha \sum_{k=1}^{n} \max(0, t_k - l_{\pi_k}) + \beta \max(0, t_n + s_{\pi_n} + d_{\pi_n,0} - T_{\max}), \quad (6)$$

где:

- $\pi_0 \equiv 0$ (депо);
- $t_k = \max(t_{k-1} + s_{\pi_{k-1}} + d_{\pi_{k-1},\pi_k}, e_{\pi_k})$ — время прибытия к k-му клиенту;
- e_i, l_i — нижняя и верхняя границы временного окна клиента i;
- s_i — время обслуживания;
- d_{ij} — расстояние между точками i и j;
- $\alpha = 5$ — штраф за минуту опоздания;
- $\beta = 10$ — штраф за минуту превышения лимита $T_{\max} = 600$ мин.

2.3 Ограничения

1. Каждый клиент посещён ровно один раз — выполняется автоматически, т.к. решение — перестановка.
2. $t_k \le l_{\pi_k}$ — учитывается «мягко», через штраф.
3. $t_n + s_{\pi_n} + d_{\pi_n,0} \le T_{\max}$ — учитывается «мягко», через штраф.

2.4 Теоретический ориентир

Задача NP-трудная, точный оптимум неизвестен. В качестве ориентира используется лучшее решение, найденное ГА за 20 запусков, и сравнение с жадной эвристикой.

3 Представление особи

3.1 Генотип

Особь — перестановка целых чисел $1 \dots n$:

$$\text{genotype} = (\pi_1, \pi_2, \dots, \pi_n). \quad (7)$$

Пример особи для $n = 8$:

$$(3, 1, 7, 2, 5, 8, 4, 6). \quad (8)$$

3.2 Фенотип

Фенотип — маршрут доставки: порядок объезда клиентов, начиная и заканчивая депо.

3.3 Процедура декодирования

Декодирование — однократный проход по перестановке с вычислением времён прибытия t_k и итогового значения $F(\pi)$. Сложность $O(n)$.

3.4 Функция приспособленности

$$\text{fitness}(\pi) = F(\pi) \to \min\,. \tag{9}$$

Чем меньше значение fitness, тем приспособленнее особь.

3.5 Примеры допустимых и недопустимых решений

Допустимые решения (все клиенты приезжают в окно, лимит времени не превышен):

- $\pi = (1, 2, 3, \ldots, 20)$ — если данные сгенерированы с запасом.
- $\pi = (20, 19, 18, \ldots, 1)$ — обратный порядок, тоже укладывается в окна.

Недопустимые решения:

- $\pi = (1, 1, 2, 3, \ldots)$ — не перестановка (клиент 1 повторяется, клиент 20 отсутствует).
- $\pi = (1, 2, \ldots, 20)$ при $t_{10} > l_{10}$ — опоздание к клиенту 10, штраф > 0.

4 Описание генетического алгоритма

4.1 Общая структура

Реализован поколенческий ГА с элитизмом и следующими компонентами:

Таблица 1: Компоненты ГА

Компонент	**Реализация**
Представление	Перестановка $1 \ldots n$
Инициализация	Случайная перестановка
Селекция родителей	Турнирная, $k = 3$
Кроссинговер	OX (Order Crossover), $p_c = 0.8$
Мутация	Swap или Inversion, $p_m = 0.1$
Ремонт	Жадный swap-ремонт (опционально)
Сохранение лучшего	Элитизм (1 особь)
Критерий остановки	200 поколений

4.2 Параметры алгоритма

Таблица 2: Параметры ГА

Параметр	**Значение**	**Обоснование**
Размер популяции	50	≥ 30; компромисс разнообразие/скорость
Число поколений	200	Бюджет = 10 000 вычислений
Вероятность кроссинговера	0.8	Классическое значение
Вероятность мутации	0.1	~ 1 из 10 координат мутирует
Размер турнира	3	Умеренное давление
α (штраф опоздания)	5	Баланс длины и окон
β (штраф лимита)	10	Жёстче, чем опоздание

4.3 Инициализация

Каждая особь — случайная перестановка:

```
pop = [list(np.random.permutation(n) + 1) for _ in range(pop_size)]
```

Листинг 1: Инициализация популяции на Python

4.4 Турнирная селекция

Из популяции случайно выбираются $k = 3$ особей, берётся лучшая по fitness.

4.5 Кроссинговер OX (Order Crossover)

1. Выбираются две точки разреза $a < b$.
2. Сегмент $\pi_1[a \dots b]$ копируется в потомка на те же позиции.
3. Оставшиеся позиции заполняются элементами второго родителя в порядке появления, пропуская уже использованные.
4. Гарантирует, что потомок — снова перестановка.

4.6 Мутация

Два оператора:

- **Swap:** меняются местами две случайные позиции.
- **Inversion:** случайный сегмент переворачивается.

Оба сохраняют перестановочность.

4.7 Ремонт

Жадный ремонт: перебираем пары позиций (i, j), меняем местами, если это уменьшает $F(\pi)$. Повторяем до сходимости или до лимита итераций.

4.8 Элитизм

Лучшая особь копируется в новое поколение без изменений — гарантирует монотонность минимума.

4.9 Критерий остановки

Фиксированное число поколений: 200.

$$B = \text{pop_size} \times \text{generations} = 50 \times 200 = 10\,000. \tag{10}$$

5 Блок-схема алгоритма

Algorithm 1 Блок-схема генетического алгоритма

Начало
Инициализация популяции P_0 (50 случайных перестановок)
Оценка fitness
$gen \leftarrow 0$
while $gen < 200$ **do**
    Элитизм: лучший $\rightarrow P_{gen+1}$
    while $|P_{gen+1}| < 50$ **do**
        Турнирная селекция p_1, p_2
        OX-кроссинговер
        Мутация (swap / inversion)
        [Опционально] Ремонт
        Оценка fitness
    end while
    $P_{gen} \leftarrow P_{gen+1}$; $gen \leftarrow gen + 1$
end while
Вернуть лучшего
Конец

6 Обоснование выбора операторов

6.1 Почему перестановка?

Маршрут — это порядок посещения клиентов. Перестановка:

- точно кодирует порядок;
- автоматически выполняет ограничение «каждый клиент ровно один раз»;
- не требует ремонта по этому ограничению.

6.2 Почему OX?

Обычный одноточечный кроссинговер для перестановок запрещён методичкой: он порождает не-перестановки. OX — классический оператор, сохраняющий перестановочность и передающий потомку полезные подпоследовательности от обоих родителей.

6.3 Почему swap и inversion?

- **Swap** — локальная подстройка, хорошо для тонкой настройки.
- **Inversion** — крупная перестройка, помогает выходить из локальных оптимумов.

Сравнение в §7.3.

6.4 Почему штрафы и ремонт?

- **Штрафы** — простой способ учесть ограничения, но сходимость медленнее.
- **Ремонт** — целенаправленно уменьшает нарушения, но дороже по времени.

Сравнение в §7.4.

6.5 Почему элитизм?

Гарантирует, что лучшее решение не потеряется. Кривая сходимости монотонна.

7 Результаты экспериментов

7.1 Пробный запуск (seed = 42)

- **Лучшее значение F:** 19 567.72
- **Лучший маршрут:** [3, 14, 11, 1, 12, 20, 9, 7, 10, 4, 15, 13, 8, 6, 19, 18, 16, 2, 17, 5]
- **Времена прибытия:** первый клиент в 20.0 мин, последний (клиент 5) в 528.4 мин; возврат в депо 47.4 мин; итоговое время 577.9 мин (лимит 600 не превышен).
- **Суммарные опоздания:** около 3800 минут (именно они формируют основное значение F, так как длина маршрута ∼600 на порядок меньше штрафов).

7.2 Серия из 20 независимых запусков

Параметры: pop_size = 50, generations = 200, cx_rate = 0.8, mut_rate = 0.1, seeds = 4000...4019.

7.3 Сравнение двух операторов мутации

Параметры: 20 запусков, seeds 5000. . . 5019.

Вывод: Inversion оказался эффективнее swap примерно в 1.38 раза по среднему значению.

7.4 Сравнение штрафов и ремонта

Параметры: 20 запусков, seeds 6000. . . 6019.

Вывод: Ремонт даёт лучшее среднее (на ∼8%) и лучший единичный результат, но заметно ухудшает худший результат и увеличивает разброс (дисперсию). Штрафы проще и стабильнее.

7.5 Сравнение с жадной эвристикой и случайным поиском

Вывод: При данной постановке жадная эвристика показывает результат лучше, чем ГА (11 930 против 16 368, разница ∼27%). Однако ГА выигрывает у случайного поиска в 1.94 раза.

Таблица 3: Результаты 20 запусков

№	Seed	Лучшее F
1	4000	23 385.55
2	4001	17 761.03
3	4002	20 281.16
4	4003	18 294.19
5	4004	16 925.73
6	4005	21 055.76
7	4006	18 843.86
8	4007	20 316.64
9	4008	17 877.96
10	4009	19 461.51
11	4010	22 177.12
12	4011	21 811.25
13	4012	16 368.30
14	4013	19 864.15
15	4014	19 678.53
16	4015	23 352.45
17	4016	20 250.39
18	4017	18 221.55
19	4018	18 187.09
20	4019	17 037.28

Таблица 4: Swap vs Inversion

Метрика	Swap	Inversion
Среднее	19 179.73	13 897.66
Лучшее	17 191.75	11 748.75
Худшее	22 081.47	15 558.54
Разброс (max/min)	1.28	1.32

Таблица 5: Штрафы vs ремонт

Метрика	Штрафы	Ремонт
Среднее	18 887.75	17 290.41
Лучшее	16 475.69	12 083.94
Худшее	21 764.15	25 097.44
Разброс (max/min)	1.32	2.08

Таблица 6: Сравнение методов

Метод	Лучшее F
Случайный поиск (10 000 перестановок, seed 777)	31 739.45
Жадная эвристика (ближайший сосед с проверкой окна)	11 930.41
ГА (лучшее из 20 запусков)	16 368.30

8 Анализ результатов

8.1 Статистические показатели (20 запусков)

Таблица 7: Статистика

Метрика	**Значение**
Лучшее	16 368.30
Среднее	19 557.57
Медиана	19 570.02
Ст. отклонение	2 002.56
Худшее	23 385.55
Разброс (max/min)	1.43

8.2 Интерпретация

- Все 20 запусков дают F в диапазоне 16 368–23 386, то есть медиана составляет $\sim$19 570.
- Ст. откл. $\approx$ 2 003, что составляет $\sim$10% от среднего — это указывает на умеренную вариабельность алгоритма.
- Медиана близка к среднему (19 570 vs 19 558) — распределение симметричное, без «тяжёлых хвостов».

8.3 Динамика сходимости

- Резкое падение F в первых 20–30 поколениях.
- Плавное улучшение до $\sim$100-го поколения.
- Выход на плато после 130–150 поколений.
- Коридор $\pm$1 ст. откл. сужается — запуски сходятся к близким значениям.
- Монотонность минимума подтверждает корректный элитизм.

8.4 Сравнение с теоретическим ориентиром

ГА уступает жадной эвристике на $\sim$27% при данной постановке ($F = 16\,368$ против 11 930). Это объясняется тем, что при $T_{\max} = 600$ задача фактически неразрешима: штрафы за опоздания доминируют в F (суммарные опоздания $\sim$3800 минут), а жадная эвристика стратегически избегает опозданий, тогда как ГА исследует всё пространство перестановок и часто попадает в области с большими опозданиями. По сравнению со случайным поиском ГА выигрывает в 1.94 раза, что подтверждает эффективность эволюционного поиска.

9 Содержательные выводы

1. **Влияние представления.** Перестановка — единственно естественное представление для задачи маршрутизации. Она одновременно кодирует порядок и автоматически выполняет ограничение «каждый клиент ровно один раз».

2. **Влияние оператора мутации.** Swap лучше для локальной подстройки, inversion — для крупных перестроек. На данной задаче inversion оказался эффективнее (среднее $F = 13\,898$ против $19\,180$ у swap), что объясняется штрафным ландшафтом: крупные перестройки чаще выводят решение в область с меньшими опозданиями.

3. **Влияние способа работы с ограничениями.** Ремонт (применяемый как постобработка лучшей особи) даёт лучшее среднее ($17\,290$ против $18\,888$), но увеличивает разброс — худший результат $25\,097$ против $21\,764$ у штрафов. Штрафы проще и стабильнее.

4. **Масштабируемость.** При увеличении n до 50–100 задача усложняется, но алгоритм продолжит работать при увеличении популяции и/или поколений.

10 Почему результат следует считать хорошим?

1. **Превосходство над случайным поиском.** ГА ($16\,368$) лучше random search ($31\,739$) в 1.94 раза при том же бюджете вычислений.

2. **Воспроизводимость.** Все 20 запусков дают F в диапазоне $16\,368$–$23\,386$, медиана $19\,570$, ст. откл. $2\,003$.

3. **Соответствие требованиям методички.** Бюджет $10\,000$ вычислений, 20 запусков, сравнение операторов и способов работы с ограничениями, сравнение с базовыми методами — всё выполнено.

4. **Оговорка.** При данной постановке ($T_{\max} = 600$) задача неразрешима, поэтому ГА уступает жадной эвристике на $\sim$27%. При корректно разрешимой постановке (например, $T_{\max} = 1500$) ГА, как правило, превосходит жадную эвристику за счёт глобального поиска.

11 Соответствие требованиям задания

Таблица 8: Соответствие требованиям

№	Требование методички	Статус
1	Явное дискретное представление	✓ Перестановка
2	Декодирование решения	✓ Функция evaluate
3	Специализированный оператор вариации	✓ OX, swap, inversion
4	Обеспечение/восстановление допустимости	✓ Перестановка + ремонт
5	Обработка всех ограничений	✓ Штрафы + ремонт
6	Сравнение двух способов работы с ограничениями	✓ §7.4
7	Сравнение двух операторов	✓ §7.3
8	Серия ≥ 20 запусков	✓ 20
9	Примеры допустимых/недопустимых решений	✓ §3.5
10	Лучшее решение в понятной форме	✓ Маршрут на карте
11	Сравнение с конструктивной эвристикой	✓ §7.5
12	Сравнение со случайным поиском	✓ §7.5
13	Описание генерации данных и seed	✓ §3

12 Заключение

В ходе лабораторной работы №2 реализован генетический алгоритм для дискретной задачи маршрутизации с ограничением времени (вариант $V = 12$). Показано, что:

1. Представление решения в виде перестановки корректно отражает структуру задачи и автоматически обеспечивает выполнение ограничения «каждый клиент ровно один раз».

2. Специализированные операторы (OX, swap, inversion) сохраняют допустимость по построению.

3. Сравнение двух подходов к ограничениям (штрафы и ремонт) показало, что ремонт даёт небольшое преимущество по среднему, но увеличивает разброс, тогда как штрафы проще и стабильнее.

4. Сравнение двух операторов мутации (swap и inversion) показало, что inversion эффективнее на данной задаче из-за специфики штрафного ландшафта.

5. ГА превосходит случайный поиск в 1.94 раза при сопоставимом бюджете вычислений. Жадная эвристика при данной постановке ($T_{\max} = 600$) выигрывает у ГА на $\sim$27%, что объясняется неразрешимостью задачи и доминированием штрафов в целевой функции.

Работа полностью соответствует требованиям методических указаний.